

인공지능수학 (2026-1)

10. 퍼셉트론 (Perceptron)

국립금오공과대학교 컴퓨터공학부
컴퓨터공학전공 / 인공지능공학전공
김 경 수

목차



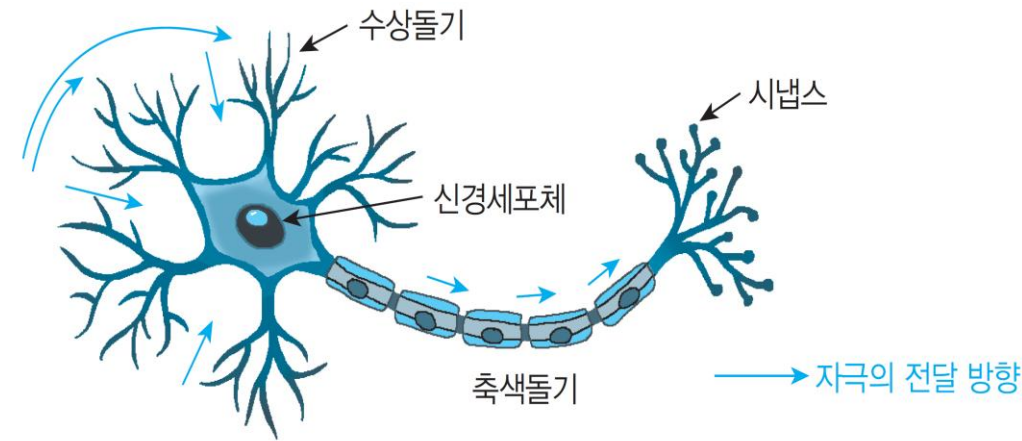
- I. 뉴런과 퍼셉트론
- II. 다층 퍼셉트론
- III. 다층 퍼셉트론의 학습

- ① 딥러닝의 가장 초기 모델 중 하나인 퍼셉트론의 정의와 구성 요소를 파악하고, 이를 수학적으로 정의할 수 있다.
- ② 단일 퍼셉트론을 이용하여 AND 연산과 OR 연산을 수행할 수 있음을 이해하고 이를 설명할 수 있다.
- ③ 단일 퍼셉트론을 이용하여 XOR 연산을 수행할 수 없는 이유를 이해하고, 이를 토대로 다층 퍼셉트론의 필요성에 대하여 설명할 수 있다.
- ④ 다층 퍼셉트론의 학습 메커니즘을 이해하고, 이를 단계별로 설명할 수 있다.
- ⑤ 간단한 다층 퍼셉트론이 주어졌을 때 모든 가중치의 업데이트 방법을 편미분을 이용해서 수학식으로 표현할 수 있다.

뉴런(neuron)과 퍼셉트론(perceptron)

뇌의 정보 전달 방법

- 뉴런(neuron)
 - 인간의 신경망의 기본 단위
 - 사람의 두뇌는 신경세포인 수없이 많은 뉴런으로 구성되며, 약 10^{10} 개의 뉴런으로 구성
- 신호 전달 메커니즘
 - ① 뉴런은 수상돌기(Dendrite)를 통하여 주변의 뉴런으로부터 신경자극을 받아 신경세포체(Cell Body)에서 어떤 처리를 하여 축색돌기(Axon)로 신호를 전달한다.
 - ② 신경세포체에서는 신호의 크기가 임계 값(threshold value)보다 큰 경우에만 시냅스(synapse)가 활성화되어 다른 뉴런으로 신호를 전달한다.



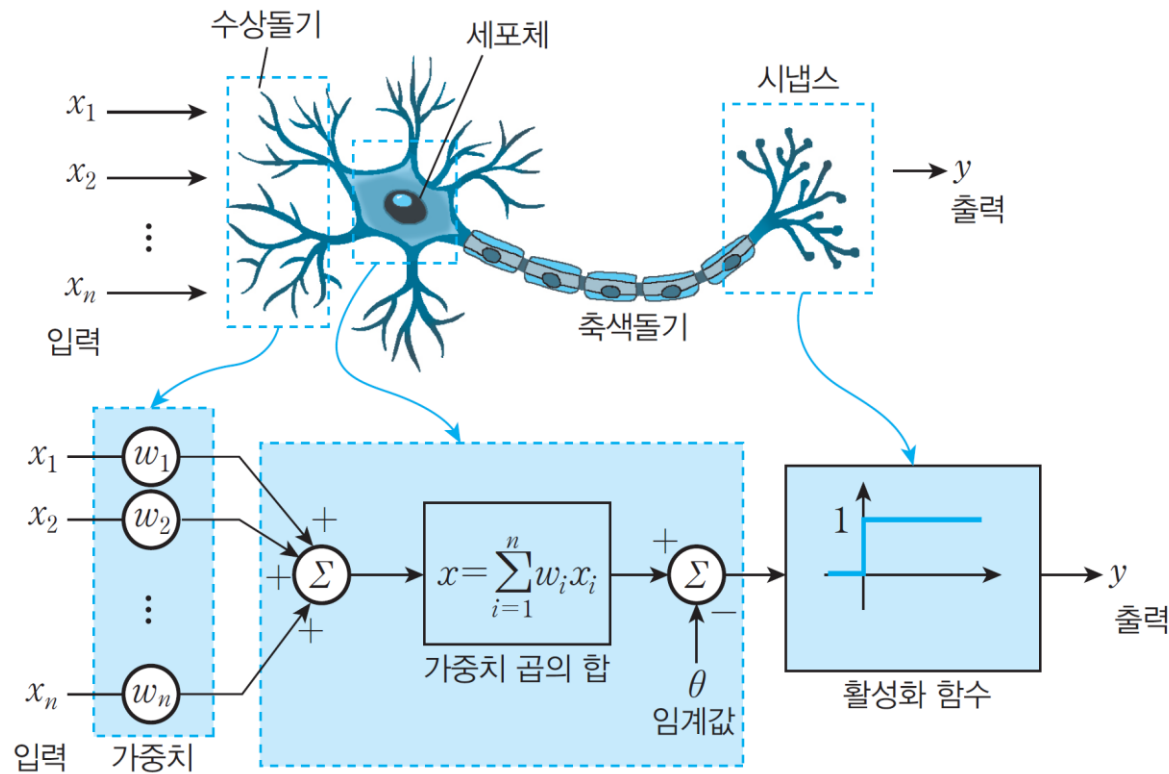
[그림 3.15] 뉴런의 구조

뇌의 정보 전달 방법

- 인간의 뇌에서 일어나는 정보 전달 방법을 모방한 것을 인공신경망(Artificial Neural Network; ANN)이라 부른다.
- 뉴런은 신경망의 기본 구성 단위인 것처럼, 인공 신경망의 기본 단위 역시 뉴런이다.
- 즉, 인공 신경망은 수많은 뉴런들의 집합체이다.
- 이때, 매우 많은 수의 뉴런들이 서로 상호 작용함으로써, 주어진 다양한 정보를 기억하고, 새로운 정보를 효과적으로 처리할 수 있다.
- 따라서, 딥러닝의 원리를 이해하기 위해서는 뉴런의 수학적 모델을 분석하는 것이 시작이다.

뉴런의 수학적 모델

- 인공신경망은 다른 뉴런으로부터 신호를 받아 합한 후에 신호의 크기가 임계 값 이상이 될 때만 출력신호를 생성하여 시냅스를 통해 다른 뉴런으로 전달한다.
- 뉴런의 수학적 모델



[그림 3.16] 뉴런의 수학적 모델

$$w_1 x_1 + w_2 x_2 + \dots + w_n x_n = \sum_{i=1}^n w_i x_i$$

$$y = \sigma \left(\sum_{i=1}^n w_i x_i \right) = \begin{cases} 1, & \text{if } x > \theta \\ 0, & \text{otherwise} \end{cases}$$

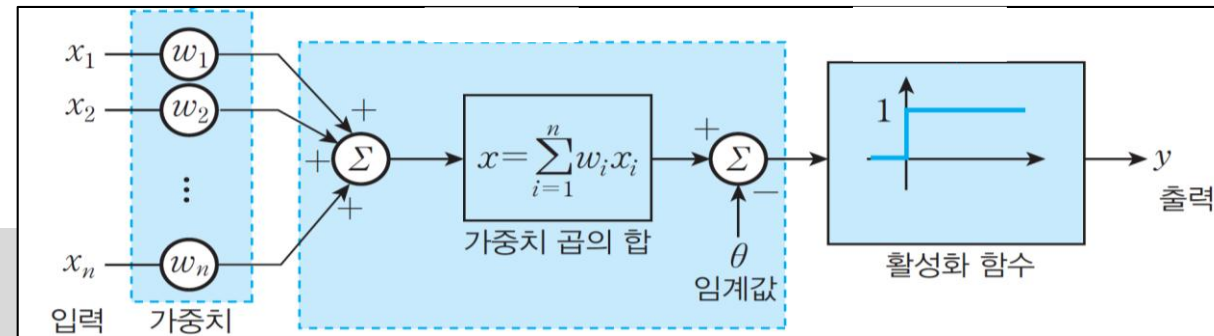
➤ 수상돌기로 들어오는 여러 신호들의 연결 강도를 가중치(Weight)로 설정한다.

뉴런의 수학적 모델

- 입력(input) : $x_0, \dots, x_n$
 - 외부로부터 자극을 전달받는 역할
- 가중치(weight) : $w_0, \dots, w_n$
 - 외부로부터 자극을 얼마만큼 받는지 강도를 결정
- 가중 합(weighted summation) : $\sum_{i=0}^n w_i x_i$
 - 입력 노드에 주어진 값과 해당 노드의 가중치를 곱한 결과를 모두 더한 값
 - 가중치 벡터를 $\vec{w} = [w_0, \dots, w_n]^T \in \mathbb{R}^{n \times 1}$, 입력 벡터를 $\vec{x} = [x_0, \dots, x_n]^T \in \mathbb{R}^{n \times 1}$ 라 할 때, 입력 벡터의 가중 합은 아래와 같이 두 벡터의 선형 조합(linear combination)과 같다.

$$\vec{w}^T \vec{x} \in \mathbb{R}$$

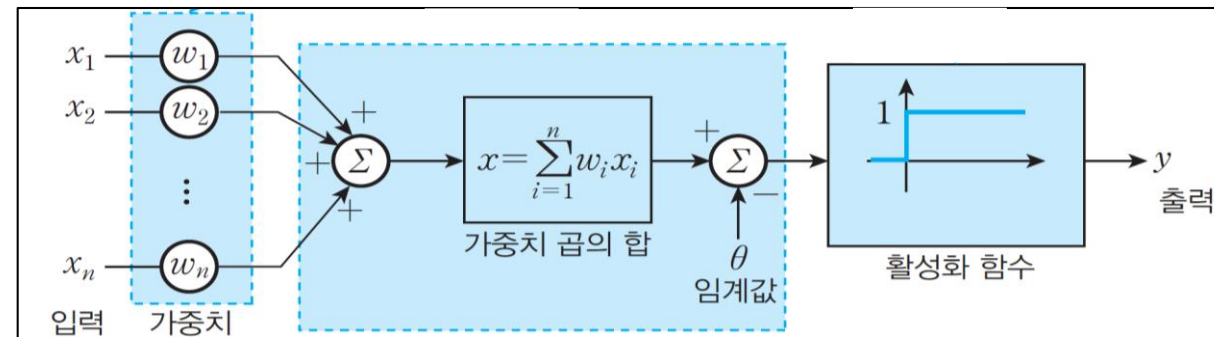
$$y = f\left(\sum_{i=1}^n w_i x_i\right) = \begin{cases} 1, & \text{if } x > \theta \\ 0, & \text{otherwise} \end{cases}$$



뉴런의 수학적 모델

- 활성화 함수(activation function) : $f(\cdot)$
 - 입력 신호의 총합을 출력 신호로 변환하는 함수
 - 가중 합 결과가 임계 값(threshold value) 이상인 경우 활성화(= 1), 아니면 비활성화(= 0)됨
 - 실제 인간의 뉴런이 출력하는 결과를 모방하여 수학적으로 표현한 것
- 출력(output) : y
 - 활성화 함수에서 변환된 최종 출력 신호(값)

$$y = f\left(\sum_{i=1}^n w_i x_i\right) = \begin{cases} 1, & \text{if } x > \theta \\ 0, & \text{otherwise} \end{cases}$$

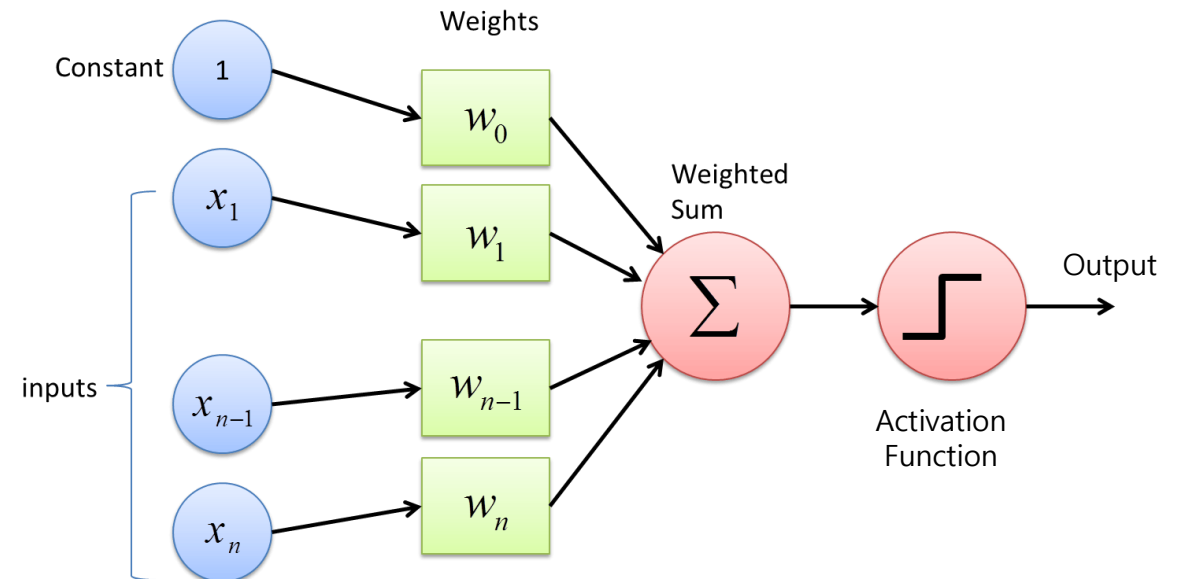
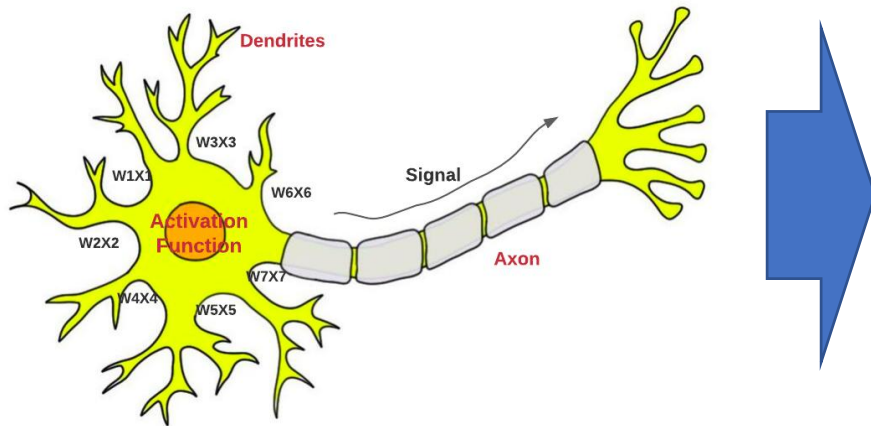


퍼셉트론(perceptron)

- 퍼셉트론(perceptron)이란?

- 미국의 심리학자 Frank Rosenblatt이 1957년에 개발한 인공 신경망의 초기 모델
- 단 하나의 뉴런만을 사용하며, 인간의 뉴런을 수학적으로 구현한 가장 기본적인 모델
- 이진 분류(binary classification)를 수행할 목적으로 고안됨

- 퍼셉트론의 기본 구조



퍼셉트론(perceptron)

- 퍼셉트론의 출력

$$y = f(w_1x_1 + \cdots + w_nx_n) = f\left(\sum_{i=1}^n w_ix_i\right)$$

- 퍼셉트론은 기본적으로 **일차 함수(=직선)**를 이용하여 분류를 수행한다.

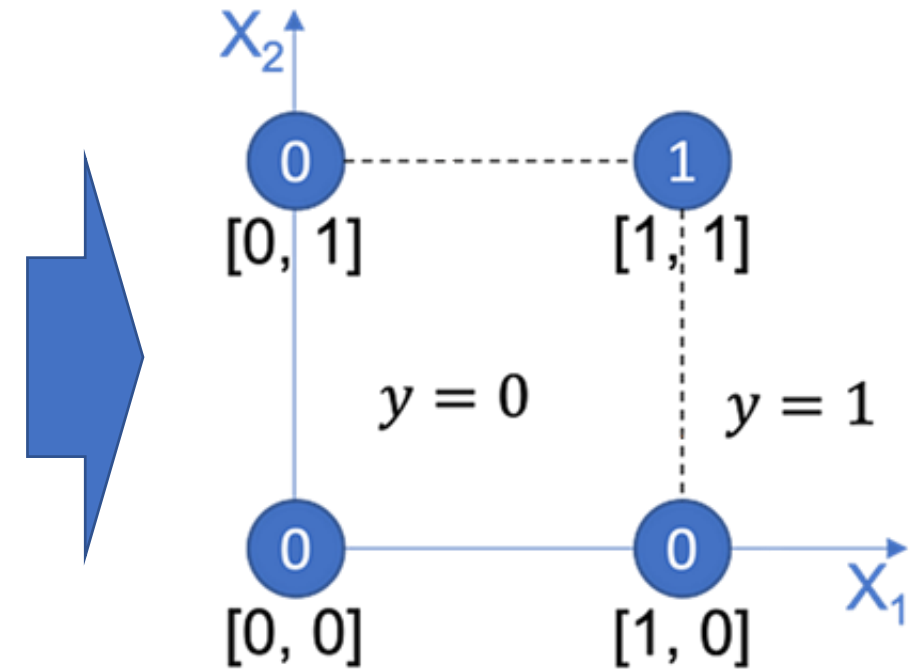
$$f\left(\sum_{i=1}^n w_ix_i\right) = \sum_{i=1}^n w_ix_i$$

- 예를 들어, 퍼셉트론의 입력 노드가 2개인 경우, 해당 퍼셉트론은 2차원 공간에서의 일차 함수를 생성한다.
- 따라서, 퍼셉트론은 **직선을 이용하여 이진 분류를 수행**하는 응용에 적합한 방법이다.
- 실제로, 이를 응용하여 AND와 OR 연산을 퍼셉트론을 이용하여 수행할 수 있다.

퍼셉트론을 활용한 AND, OR 연산 수행

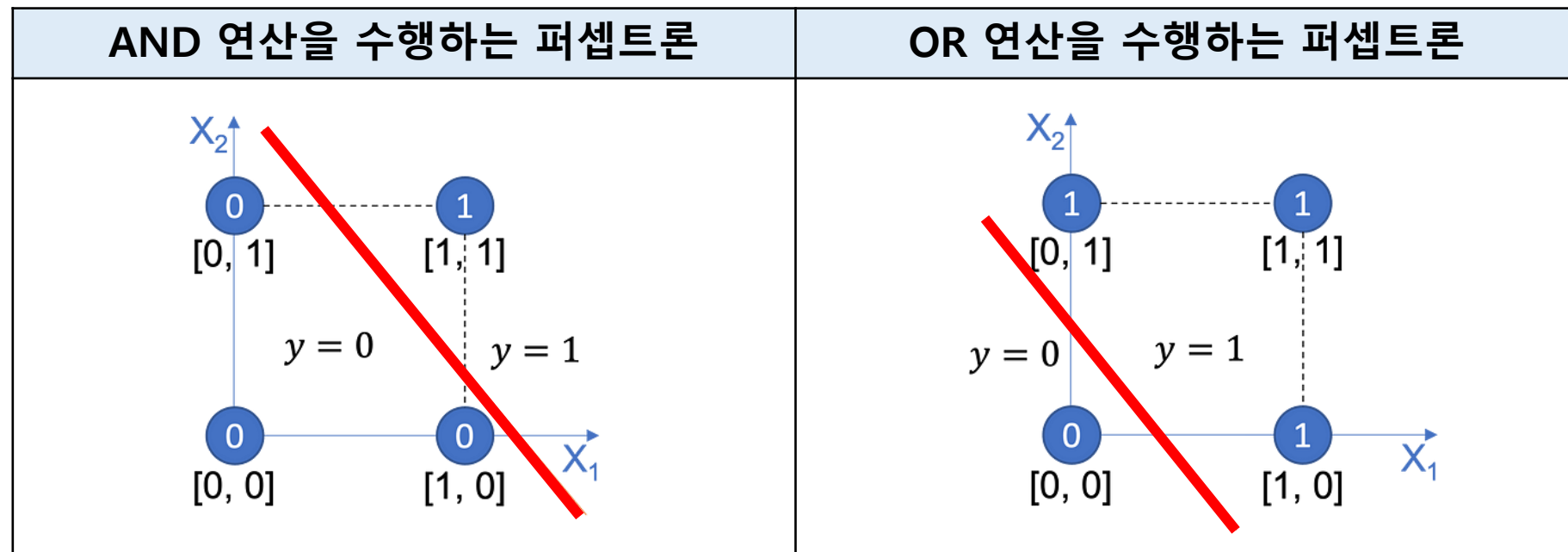
- x_1 과 x_2 가 주어졌을 때 AND, OR, XOR 연산 결과를 2차원 평면상에 표현하면 다음과 같다.

x_1	x_2	$x_1 \wedge x_2$	$x_1 \vee x_2$	$x_1 \text{ XOR } x_2$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



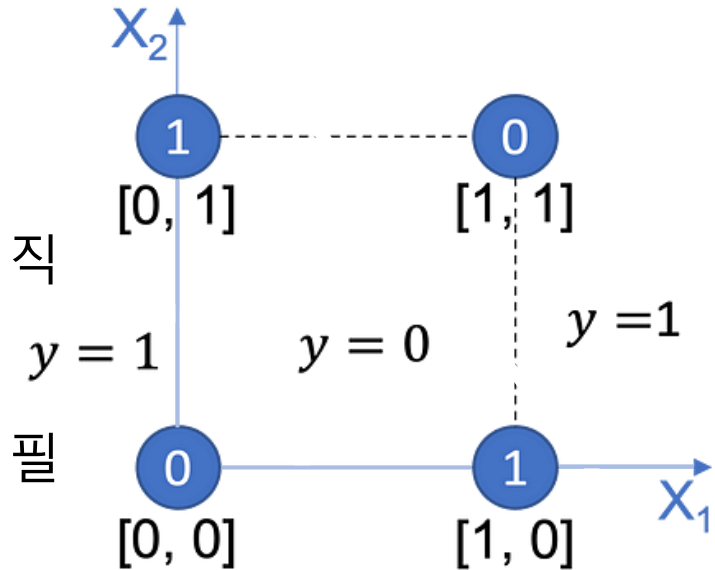
퍼셉트론을 활용한 AND, OR 연산 수행

- 퍼셉트론을 이용하면, 아래와 같이 AND와 OR 연산을 수행할 수 있다.
 - 퍼셉트론을 사용하면 아래와 같이 2차원 평면 상에서 AND와 OR 연산 결과를 구분할 수 있는 일차 함수(= 직선)를 그릴 수 있다.



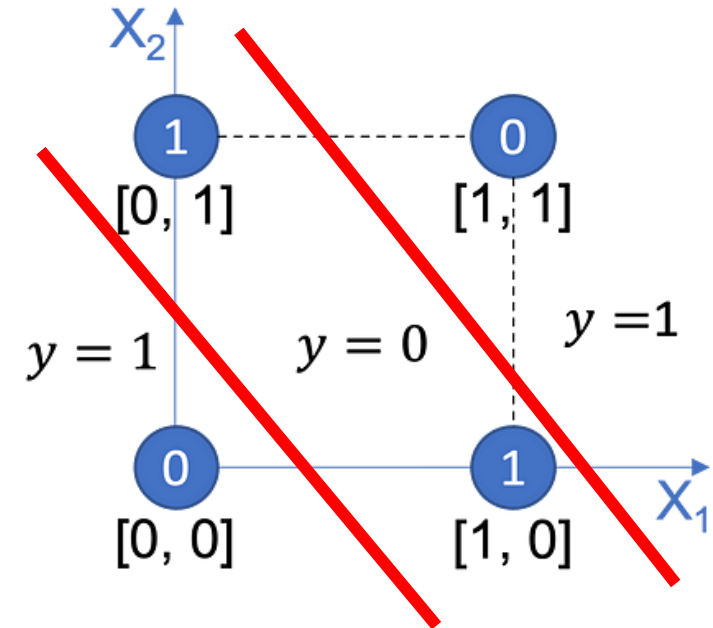
퍼셉트론의 한계

- 그러나 퍼셉트론은 XOR (Exclusive-OR) 연산을 수행할 수 없다.
 - 1969년 Marvin Minsky와 Seymour Papert에 의해서 밝혀짐.
 - XOR 연산 결과를 2차원 상에 표현하면 오른쪽 그림과 같다.
 - 직선 1개만으로는 XOR 연산 결과(0과 1)를 정확하게 분리하는 직선을 그릴 수 없다.
 - XOR 연산 결과를 정확하게 분리하기 위해서는 최소 직선 2개가 필요하다.



퍼셉트론의 한계

- 이를 보완하기 위해서는 어떻게 해야 하는가?
 - 퍼셉트론은 일차 함수(직선) 1개를 표현할 수 있다.
 - 따라서, 퍼셉트론을 여러 개 만들고, 이들을 하나로 결합한다.
 - 이를 통해, 복잡한 데이터에 대해서도 분류를 수행할 수 있다.
- 이와 같이 여러 개의 퍼셉트론을 하나로 연결하여 구현한 퍼셉트론을 “다층 퍼셉트론(multi-layer perceptron)” 이라고 한다.



퍼셉트론 예제

예제 3.11

3개의 입력을 가진 퍼셉트론에서 입력신호가 각각 $x_1=1, x_2=1, x_3=0$ 이고, 각 입력에 대한 가중치가 $w_1=2, w_2=3, w_3=4$ 라고 가정한다. 만일 활성화 함수 $f(\cdot)$ 가 다음과 같이 주어질 때 출력신호 y 를 계산하라.

$$f(x) = \begin{cases} 1, & x > 4 \\ 0, & x \leq 4 \end{cases}$$

풀이

먼저 가중치의 곱을 구하여 합하면 식(26)으로부터 가중치 곱의 합은 다음과 같다.

$$x = w_1 x_1 + w_2 x_2 + w_3 x_3 = 2 + 3 + 0 = 5$$

퍼셉트론의 출력 y 는 식(27)로부터 $\theta = 4$ 인 경우이므로

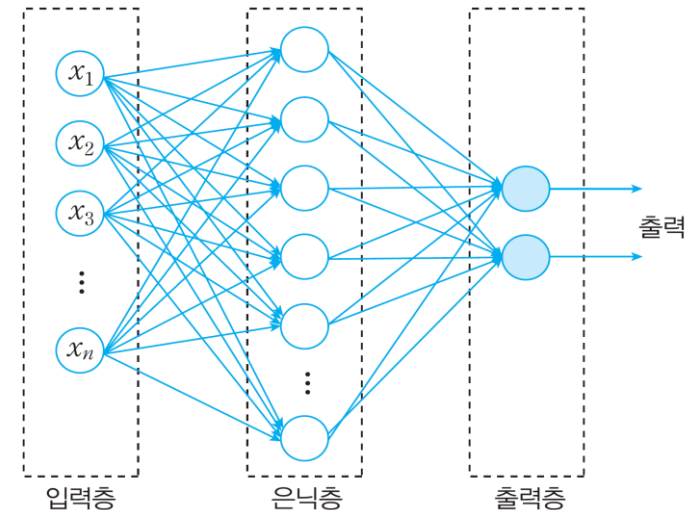
$$y = f(x - 4) = f(5 - 4) = f(1) = 1$$

이 얻어진다.

다층 퍼셉트론

다층 퍼셉트론이란?

- 다층 퍼셉트론이란, 두 개 이상의 퍼셉트론을 연결하여 구성한 퍼셉트론 모델로, 입력층과 출력층 사이에 **은닉층(Hidden Layer)**이 존재한다.
- 다층 퍼셉트론의 구성
 - 단일 퍼셉트론을 여러 개 연결함으로써 구현할 수 있다.
 - 따라서, 다층 퍼셉트론은 단일 퍼셉트론보다 더 많은 정보를 저장 및 표현할 수 있다.
- 다층 퍼셉트론에서의 이슈(issue)
 - 어떻게 수많은 가중치를 학습시킬 수 있는가?
 - 이는 오차 역전파 알고리즘을 이용하여 수행할 수 있다.



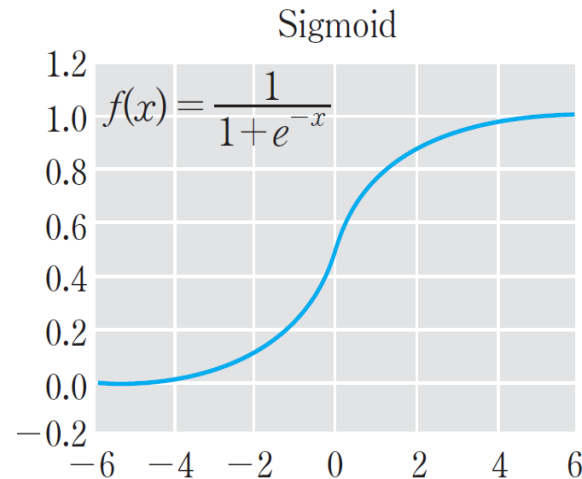
[그림 3.18] 다층 퍼셉트론의 구조

다층 퍼셉트론의 특성

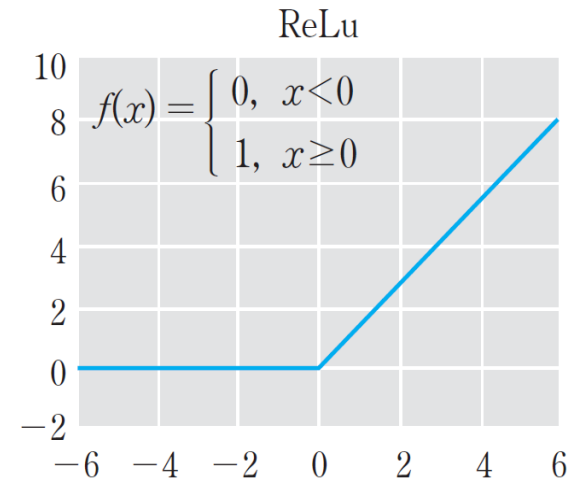
- 1970년부터 다층 퍼셉트론을 학습시킬 수 있는 오차 역전파 학습알고리즘이 개발됨
 - 이에 따라, 1980년대부터 인공 신경망에 대한 연구가 다시 활성화됨
- 오차 역전파 알고리즘을 이용한 학습 순서
 - ① 입력이 주어지면 순방향(입력 층 → 출력 층)으로 신호를 순차적으로 전파(propagation)하여 최종 출력 값을 계산한다.
 - ② 이후, 실제 출력 결과와 원하는 출력(= 정답 or Label) 간의 차이인 오차(Error 또는 Loss)를 정의한다.
 - ③ 이 오차를 다시 역방향(출력 층 → 입력 층)으로 전파(backpropagation)하면서 오차를 감소시키는 방향으로 가중치(Weights)를 업데이트한다.

다층 퍼셉트론의 특성

- 퍼셉트론은 계단함수(step function)를 활성화 함수로 사용하지만 다층 퍼셉트론에서는 다양한 형태의 비선형(nonlinear) 함수를 활성화 함수로 사용한다.
- 계단 함수가 활성화 함수로 잘 사용되지 않는 이유
 - 미분(differential)이 불가능하므로, 오차 역전파 알고리즘을 활용할 수 없음
 - 오차 역전파 알고리즘을 이용하기 위해서는 미분이 가능해야 함



(a) 시그모이드 함수



(b) 렐루 함수

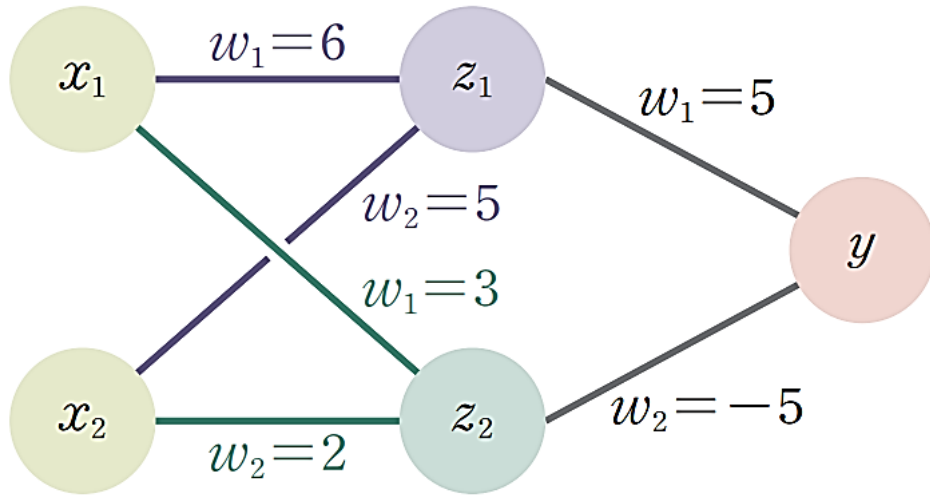
[그림 3.20] 전형적인 비선형 활성화 함수

다층 퍼셉트론의 표현력

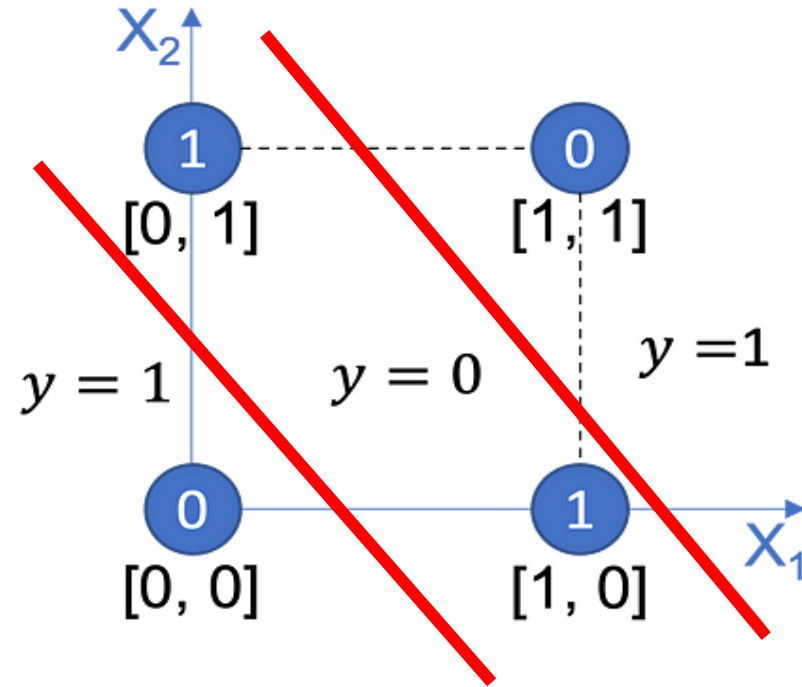
- 두 개 이상의 퍼셉트론을 결합하여 모델을 구성함에 따라, 정보를 저장하는 가중치 (weight)가 증가하여 보다 복잡한 문제에 대해서도 표현할 수 있다.
- 실제로, 다층 퍼셉트론을 이용하면 XOR 문제를 해결할 수 있다.
 - XOR 문제는 직선 두 개를 이용하여 분류가 가능한데, 다층 퍼셉트론을 사용하면 직선을 두 개 이상 표현할 수 있으므로 XOR 문제를 해결할 수 있음.

다층 퍼셉트론의 표현력

- XOR 문제 해결을 위한 다층 퍼셉트론의 구조의 예(example)



활성화 함수 $f(x) = \begin{cases} 0 & \text{if } \theta < 4 \\ 1 & \text{otherwise} \end{cases}$



다층 퍼셉트론의 학습

다층 퍼셉트론의 학습 메커니즘

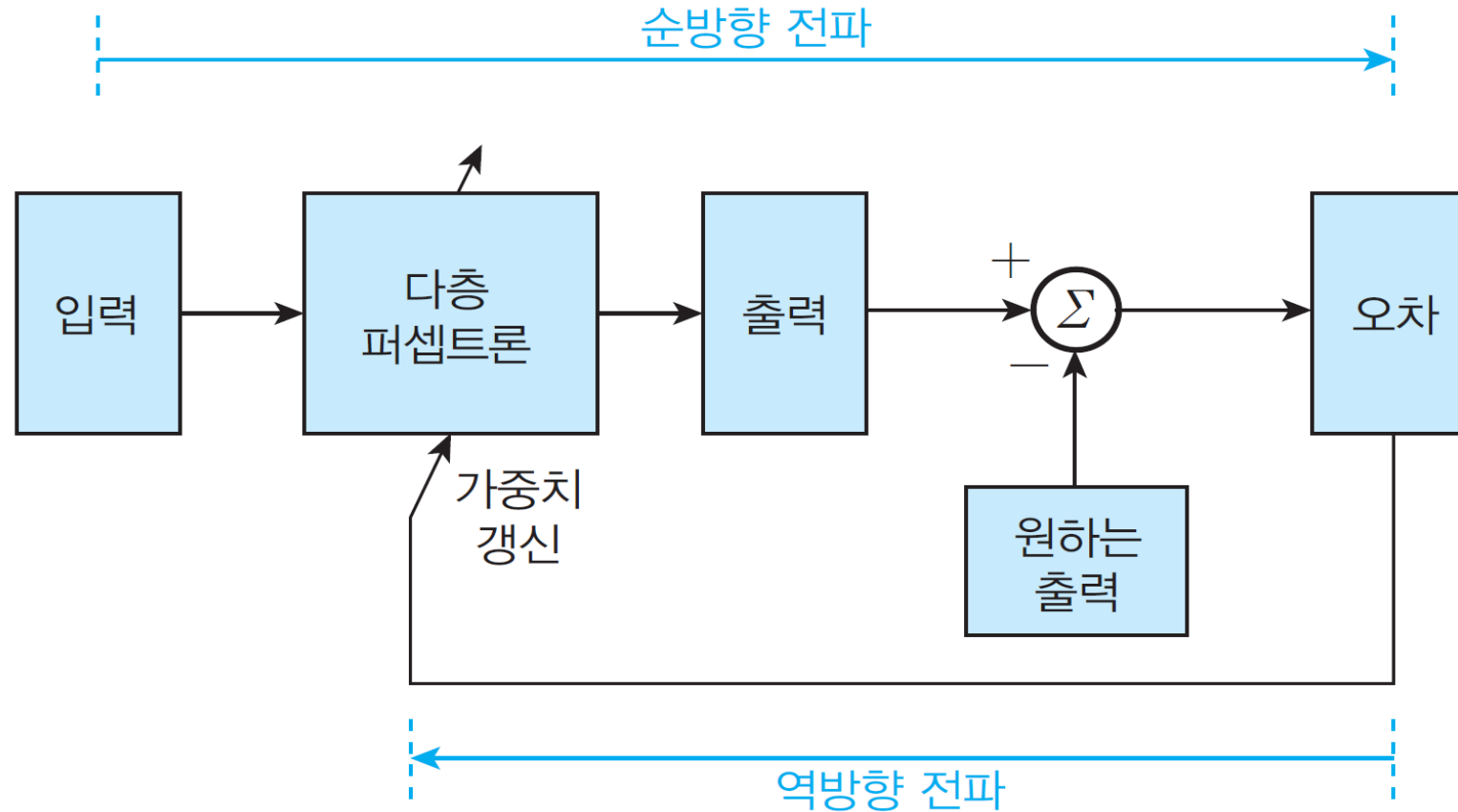
- 우리의 목표

- 주어진 입력 데이터에 대하여 다층 퍼셉트론의 출력 결과와 해당 입력 데이터의 실제 정답 값과의 오차(error)를 최소화 할 수 있는 가중치 값들을 찾는다.

- 이를 위해서 수행해야 할 작업

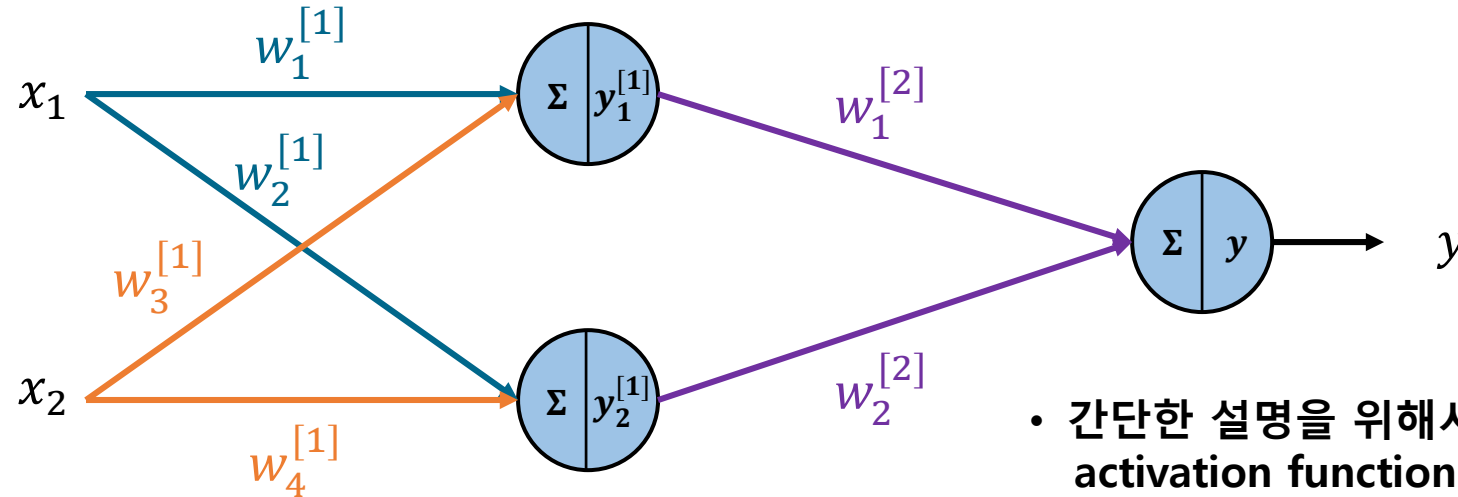
- ① 입력 데이터에 대한 퍼셉트론의 출력 결과 확인
 - ② 해당 입력 데이터의 실제 정답 값과 퍼셉트론의 출력 값과의 오차 계산하기
 - ③ 오차를 최소화하기
 - ④ 해당 오차를 최소화할 수 있도록 기존에 부여된 가중치의 값들을 수정하기

다층 퍼셉트론의 학습 메커니즘



[그림 3.19] 역전파 학습알고리즘의 개념도

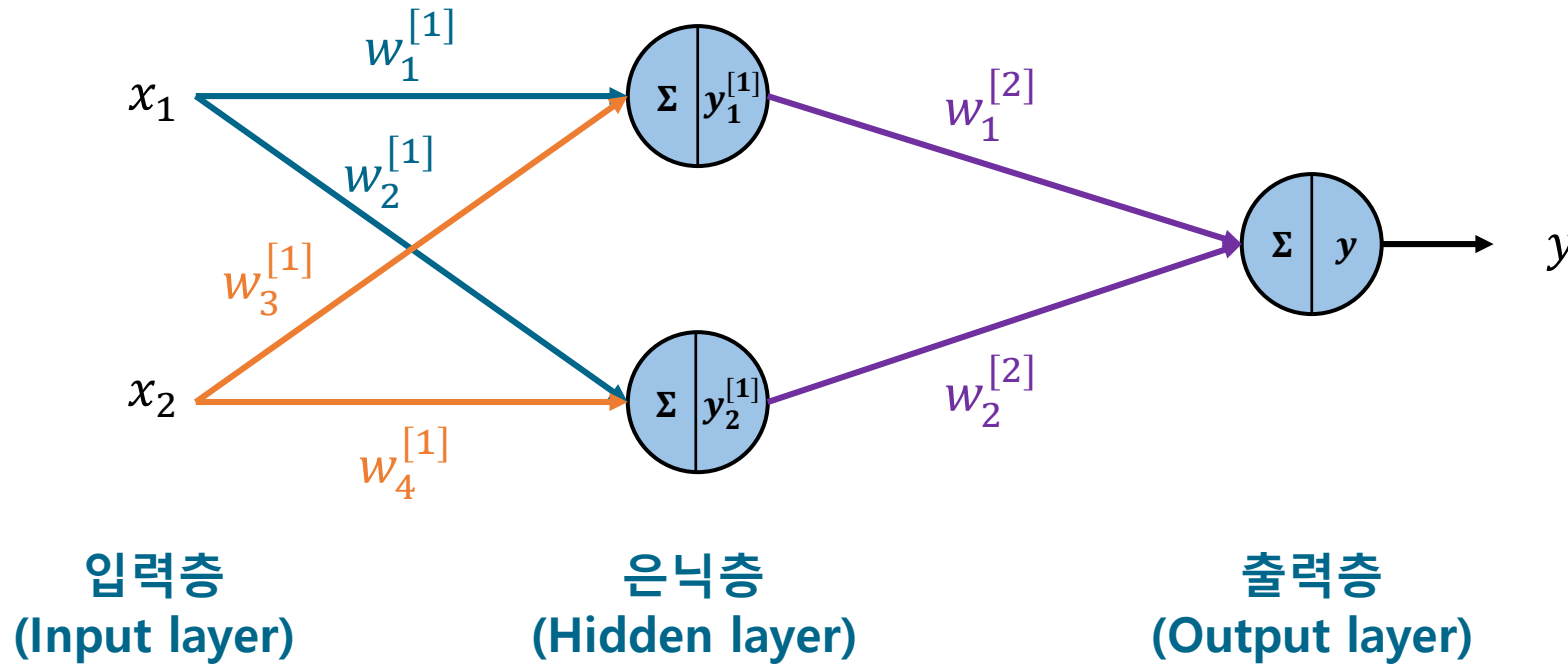
다층 퍼셉트론의 학습 메커니즘 예



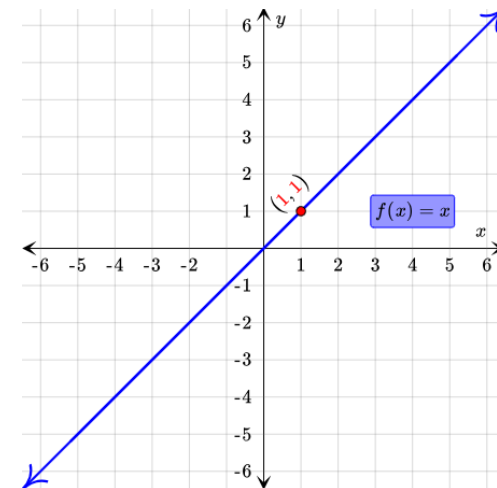
• 간단한 설명을 위해서 본 예제에서는 activation function으로 $f(x) = x$ 를 사용함.

- 위와 같은 다층 퍼셉트론을 가정해 보자.
 - 우리의 목표는 학습 데이터가 입력으로 주어졌을 때, 해당 학습 데이터를 가장 잘 표현할 수 있도록 가중치 6개의 최적의 값을 찾는 것이다.
 - 이를 위해, 먼저 각각의 가중치들을 임의의 값으로 초기화(initialization)한다.
 - 그 다음 학습 데이터 $\vec{x} = [x_1, x_2]^T$ 가 입력으로 주어진다.

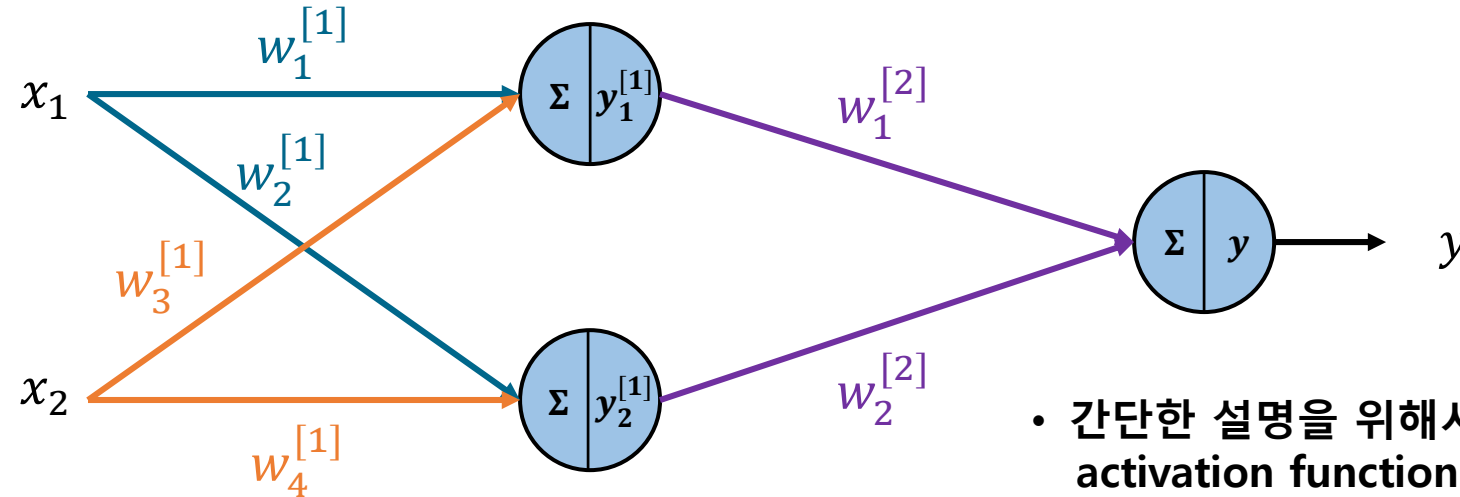
다층 퍼셉트론의 학습 메커니즘 예



- 본 예제에서 사용하는 활성화 함수 $f(x) = x$ 는 항등함수(identity function)라고 부른다.



① 각 노드(neuron)별 출력 값을 순차적으로 계산



• 간단한 설명을 위해서 본 예제에서는 activation function으로 $f(x) = x$ 를 사용함.

- 학습 데이터 $\vec{x} = [x_1, x_2]^T$ 가 주어짐에 따라, $y_1^{[1]}, y_2^{[1]}, y$ 를 순차적으로 계산한다.

$$y_1^{[1]} = f(x_1 w_1^{[1]} + x_2 w_3^{[1]}) = x_1 w_1^{[1]} + x_2 w_3^{[1]}$$

$$y_2^{[1]} = f(x_1 w_2^{[1]} + x_2 w_4^{[1]}) = x_1 w_2^{[1]} + x_2 w_4^{[1]}$$

$$y = f(y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]}) = y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]}$$

② 출력 값의 오차 계산

※ 오차 함수는 손실 함수(loss function)라고도 부른다.

- 학습 데이터 $\vec{x} = [x_1, x_2]^T$ 에 대한 출력 결과 y 가 계산되었다.
- 이때, 학습 데이터 $\vec{x}$ 의 정답 값(ground-truth)을 $\hat{y}$ 라 하자.
- 다층 퍼셉트론의 출력 결과와 실제 정답 값 사이의 오차를 계산하기 위해 아래와 같이 오차 함수(error function)가 사용된다.

$$J(y, \hat{y}) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- 상기 오차 함수는 i 번째 입력 데이터($= \vec{x}_i$)에 대한 다층 퍼셉트론의 출력 결과($= y_i$)와 해당 입력 데이터의 정답 출력 값($= \hat{y}_i$) 사이의 오차(error)를 계산한 결과이다.
- 즉, n 개의 학습(입력) 데이터들의 오차의 평균을 계산한 것이 바로 위의 오차 함수이다.
- 상기 오차 함수에서 상수 $1/2$ 은 계산의 편의를 위해서 추가한 것이며 없어도 무방함.

② 출력 값의 오차 계산

- 본 예제에서는 학습 데이터가 1개라고 가정한다.
- 그렇다면, 앞서 확인한 출력 결과의 오차를 오차 함수를 이용하여 다음과 같이 표현할 수 있다.

$$\begin{aligned} J(y, \hat{y}) &= \frac{1}{2} (y - \hat{y})^2 \\ &= \frac{1}{2} \left(f \left(y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} \right) - \hat{y} \right)^2 \\ &= \frac{1}{2} \left(y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y} \right)^2 \end{aligned}$$

- 간단한 설명을 위해서 본 예제에서는 **activation function**으로 $f(x) = x$ 를 사용함.

③ 오차의 최소화 방법

- 앞의 오차 함수를 다시 살펴보자.

$$J(y, \hat{y}) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- 상기 오차 함수는 이차 함수(quadratic function)이다.
- 이차 함수 $y = ax^2 + b$ 를 최소화하는 해를 찾는 방법
 - ✓ 기울기가 0이 되는 해 x^* 를 찾는다.
 - ✓ 기울기가 0이 되는 해 x^* 를 찾는 방법 → 도함수($\frac{dy}{dx} = 0$)를 활용하면 찾을 수 있다.
- 우리의 목표는 상기 오차 함수를 최소화할 수 있는 최적의 가중치 값을 찾는 것이다.
- 따라서, 상기 오차 함수를 6개의 가중치 $w_1^{[1]}, \dots, w_2^{[2]}$ 각각에 대하여 편미분(partial differential)한 도함수들을 이용하여 가중치를 갱신할 수 있다.

③ 오차의 최소화 방법

- 오차 함수의 편미분 방법

➤ 앞서 설명한 입력 데이터 1개에 대한 오차 함수를 각각의 가중치에 대해서 편미분한다.

$$\begin{aligned}\frac{\partial}{\partial w_{21}^{[2]}} J(y, \hat{y}) &= \frac{\partial}{\partial w_{21}^{[2]}} \left[\frac{1}{2} (\mathbf{y} - \hat{y})^2 \right] \\ &= \frac{\partial}{\partial w_{21}^{[2]}} \left[\frac{1}{2} \left(\mathbf{y}_1^{[1]} w_1^{[2]} + \mathbf{y}_2^{[1]} w_2^{[2]} - \hat{y} \right)^2 \right]\end{aligned}$$

- 위 오차 함수는 여러 개의 가중치로 이루어진 다변수 함수(multivariable function)이다.
- 편미분은 다변수 함수에서 특정 변수에 대해서만 미분을 수행하는 방법이다.
- 미분하려는 변수를 제외한 나머지 변수들은 모두 상수(constant)로 가정하고, 미분을 수행한다.

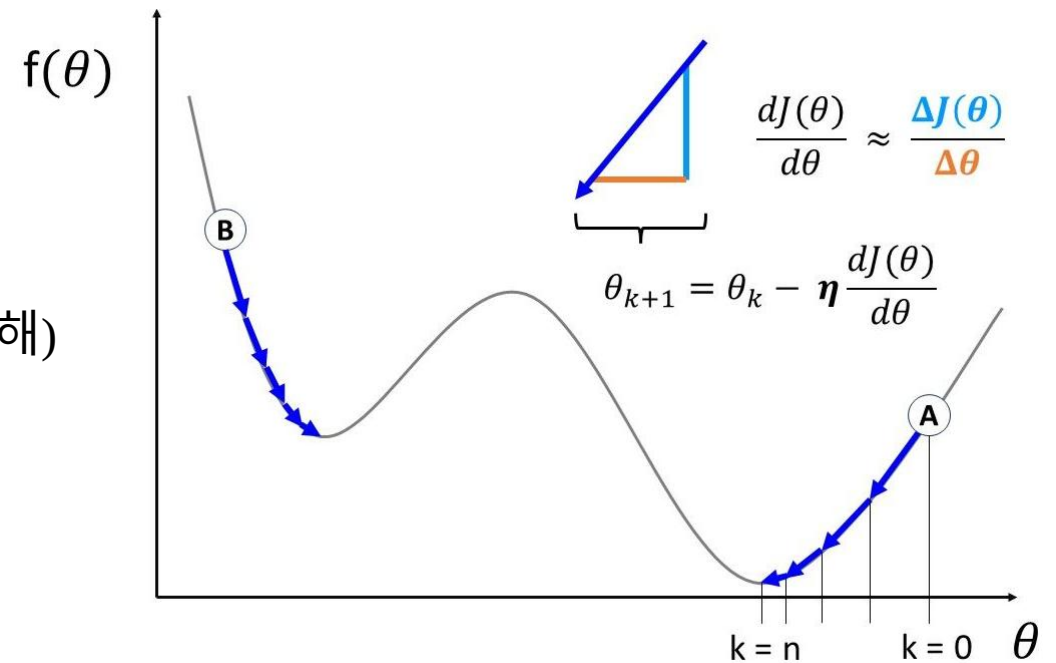
③ 오차의 최소화 방법

- 경사 하강법(gradient descent)

- 함수를 최소(또는 최대)화 할 수 있는 최적의 해(optimal solution)를 탐색하는 알고리즘
- 함수의 기울기(=도함수)를 이용하여 기울기가 최소가 되는 지점을 향해서 해를 점진적으로 갱신하면서 최적의 해를 탐색한다.
- 경사 하강법의 공식과 원리

$$\theta_{k+1} = \theta_k - \eta \frac{d}{d\theta} J(\theta)$$

- θ_{k+1} : 다음 시점에서의 최적 해(= 갱신된 최적 해)
- θ_k : 현재 시점에서의 최적 해
- η : 학습률 (learning rate)
- $dJ(\theta)/d\theta$: 함수 $J(\theta)$ 의 도함수



③ 오차의 최소화 방법

- [참고] 경사 하강법 의사코드

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 출력층부터 입력층 방향으로 가중치를 **역순**으로 갱신(update)한다.(= **backpropagation**)
- 이때, 가중치의 갱신을 위해 경사 하강법 공식이 활용된다.
- 경사 하강법을 이용한 가중치 갱신 방법

$$w_{t+1} \leftarrow w_t - \eta \frac{\partial}{\partial} J(y, \hat{y})$$

- 이때, 우리의 목표는 오차 함수를 최소화 할 수 있는 최적의 가중치를 찾는 것이므로 오차 함수 $J(y, \hat{y})$ 를 “가중치에 대한 함수”로 다루어야 한다.
- 오차 함수 $J(y, \hat{y})$ 를 자세히 살펴보면 모든 6개의 가중치를 모두 포함하고 있다. 따라서, 오차 함수 $J(y, \hat{y})$ 를 갱신하려는 가중치에 대하여 편미분할 수 있다.
- 학습률 η 은 일반적으로 매우 작은 값(e.g., 0.001)으로 설정한다.

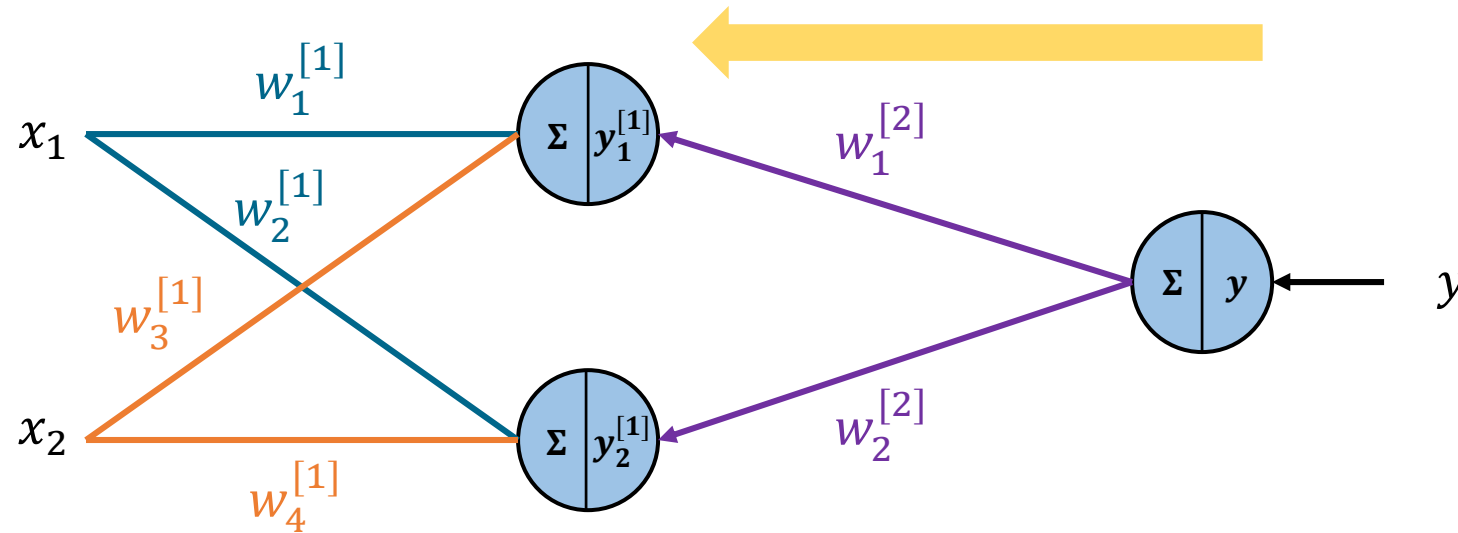
④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수의 분석

$$\begin{aligned}
 J(y, \hat{y}) &= \frac{1}{2} (y - \hat{y})^2 \\
 &= \frac{1}{2} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y})^2 \\
 &= \frac{1}{2} \left((x_1 w_1^{[1]} + x_2 w_3^{[1]}) w_1^{[2]} + (x_1 w_2^{[1]} + x_2 w_4^{[1]}) w_2^{[2]} - \hat{y} \right)^2 \\
 &= \frac{1}{2} \left(x_1 w_1^{[1]} w_1^{[2]} + x_2 w_3^{[1]} w_1^{[2]} + x_1 w_2^{[1]} w_2^{[2]} + x_2 w_4^{[1]} w_2^{[2]} - \hat{y} \right)^2
 \end{aligned}$$

$$\begin{aligned}
 y_1^{[1]} &= f(x_1 w_1^{[1]} + x_2 w_3^{[1]}) = x_1 w_1^{[1]} + x_2 w_3^{[1]} \\
 y_2^{[1]} &= f(x_1 w_2^{[1]} + x_2 w_4^{[1]}) = x_1 w_2^{[1]} + x_2 w_4^{[1]} \\
 y &= f(y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]}) = y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]}
 \end{aligned}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신



$$w_1^{[2]*} \leftarrow w_1^{[2]} - \lambda \frac{\partial}{\partial w_1^{[2]}} J(y, \hat{y})$$

$$w_2^{[2]*} \leftarrow w_2^{[2]} - \lambda \frac{\partial}{\partial w_2^{[2]}} J(y, \hat{y})$$



$$w_1^{[2]*} \leftarrow w_1^{[2]} - \lambda (y - \hat{y}) y_1^{[1]}$$

$$w_2^{[2]*} \leftarrow w_2^{[2]} - \lambda (y - \hat{y}) y_2^{[1]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_1^{[2]}$ 와 $w_2^{[2]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\begin{aligned}\frac{\partial}{\partial w_1^{[2]}} J(y, \hat{y}) &= \frac{\partial}{\partial w_1^{[2]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) \\ &= (y - \hat{y}) \times \frac{\partial}{\partial w_1^{[2]}} (y - \hat{y}) \\ &= (y - \hat{y}) \times \frac{\partial}{\partial w_1^{[2]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y}) \\ &= (y - \hat{y}) \times y_1^{[1]} \\ &= y_1^{[1]^2} w_1^{[2]} + y_1^{[1]} y_2^{[1]} w_2^{[2]} - y_1^{[1]} \hat{y}\end{aligned}$$

$$\therefore \frac{\partial}{\partial w_1^{[2]}} J(y, \hat{y}) = (y - \hat{y}) y_1^{[1]}$$

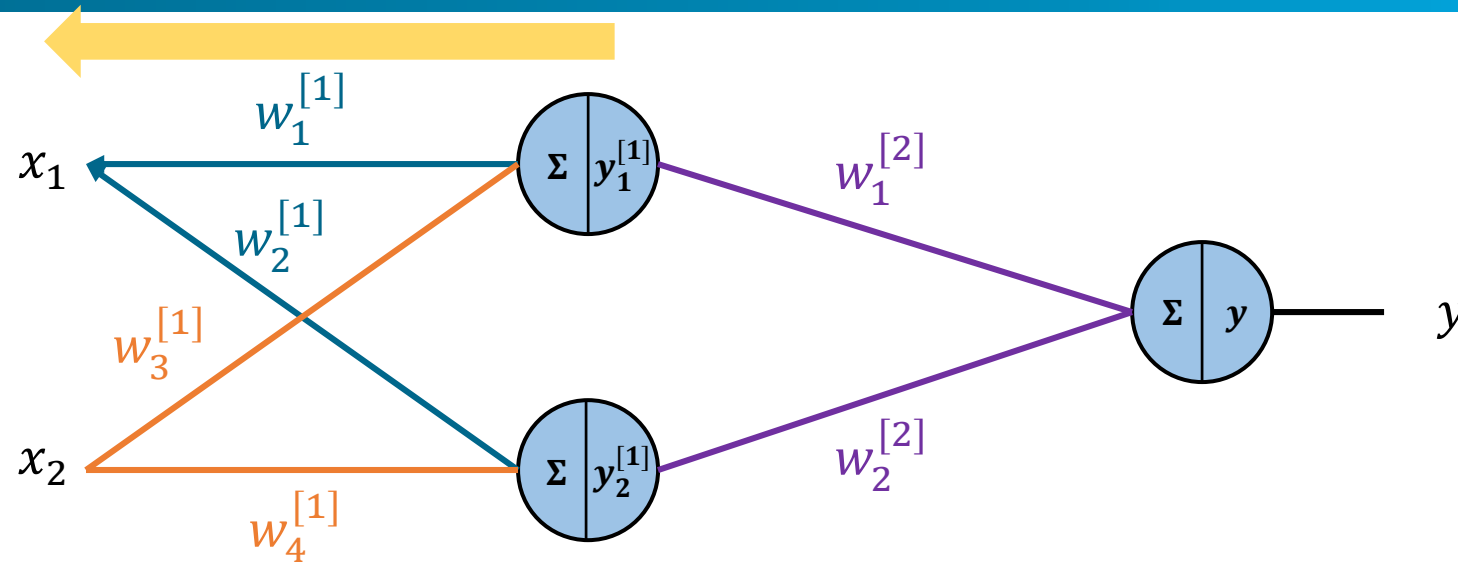
④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_1^{[2]}$ 와 $w_2^{[2]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\begin{aligned}\frac{\partial}{\partial w_2^{[2]}} J(y, \hat{y}) &= \frac{\partial}{\partial w_2^{[2]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) \\&= (y - \hat{y}) \times \frac{\partial}{\partial w_2^{[2]}} (y - \hat{y}) \\&= (y - \hat{y}) \times \frac{\partial}{\partial w_2^{[2]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y}) \\&= (y - \hat{y}) \times y_2^{[1]} \\&= y_1^{[1]} y_2^{[1]} w_1^{[2]} + y_2^{[1]^2} w_2^{[2]} - \hat{y} y_2^{[1]}\end{aligned}$$

$$\therefore \frac{\partial}{\partial w_2^{[2]}} J(y, \hat{y}) = (y - \hat{y}) y_2^{[1]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신



$$w_1^{[1]*} \leftarrow w_1^{[1]} - \lambda \frac{\partial}{\partial w_1^{[1]}} J(y, \hat{y})$$

$$w_2^{[1]*} \leftarrow w_2^{[1]} - \lambda \frac{\partial}{\partial w_2^{[1]}} J(y, \hat{y})$$



$$w_1^{[1]*} \leftarrow w_1^{[1]} - \lambda (y - \hat{y}) x_1 w_1^{[2]}$$

$$w_2^{[1]*} \leftarrow w_2^{[1]} - \lambda (y - \hat{y}) x_1 w_2^{[2]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_1^{[1]}$ 과 $w_2^{[1]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\frac{\partial}{\partial w_1^{[1]}} J(y, \hat{y}) = \frac{\partial}{\partial w_1^{[1]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) = (y - \hat{y}) \times \frac{\partial}{\partial w_1^{[1]}} (y - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_1^{[1]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_1^{[1]}} (x_1 w_1^{[1]} w_1^{[2]} + x_2 w_3^{[1]} w_1^{[2]} + x_1 w_2^{[1]} w_2^{[2]} + x_2 w_4^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times x_1 w_1^{[2]}$$

$$= x_1 y_1^{[1]} w_1^{[2]^2} + x_1 y_2^{[1]} w_1^{[2]} w_2^{[2]} - x_1 \hat{y} w_1^{[2]}$$

$$\therefore \frac{\partial}{\partial w_1^{[1]}} J(y, \hat{y}) = (y - \hat{y}) x_1 w_1^{[2]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_1^{[1]}$ 과 $w_2^{[1]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\frac{\partial}{\partial w_2^{[1]}} J(y, \hat{y}) = \frac{\partial}{\partial w_2^{[1]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) = (y - \hat{y}) \times \frac{\partial}{\partial w_2^{[1]}} (y - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_2^{[1]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y})$$

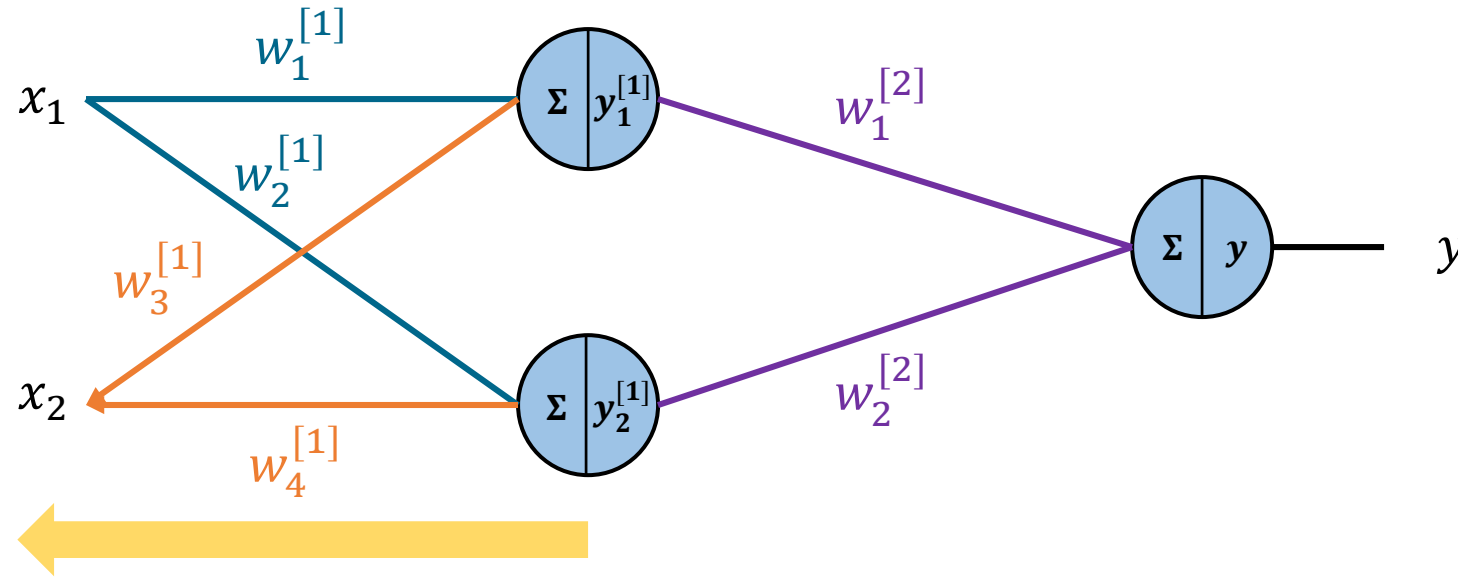
$$= (y - \hat{y}) \times \frac{\partial}{\partial w_2^{[1]}} (x_1 w_1^{[1]} w_1^{[2]} + x_2 w_3^{[1]} w_1^{[2]} + x_1 w_2^{[1]} w_2^{[2]} + x_2 w_4^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times x_1 w_2^{[2]}$$

$$= x_1 y_1^{[1]} w_1^{[2]} w_2^{[2]} + x_1 y_2^{[1]} w_2^{[2]^2} - x_1 \hat{y} w_2^{[2]}$$

$$\therefore \frac{\partial}{\partial w_2^{[1]}} J(y, \hat{y}) = (y - \hat{y}) x_1 w_2^{[2]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신



$$w_3^{[1]*} \leftarrow w_3^{[1]} - \lambda \frac{\partial}{\partial w_3^{[1]}} J(y, \hat{y})$$

$$w_4^{[1]*} \leftarrow w_4^{[1]} - \lambda \frac{\partial}{\partial w_4^{[1]}} J(y, \hat{y})$$



$$w_3^{[1]*} \leftarrow w_3^{[1]} - \lambda(y - \hat{y})x_2w_1^{[2]}$$

$$w_4^{[1]*} \leftarrow w_4^{[1]} - \lambda(y - \hat{y})x_2w_2^{[2]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_3^{[1]}$ 과 $w_4^{[1]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\frac{\partial}{\partial w_3^{[1]}} J(y, \hat{y}) = \frac{\partial}{\partial w_3^{[1]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) = (y - \hat{y}) \times \frac{\partial}{\partial w_3^{[1]}} (y - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_3^{[1]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_3^{[1]}} (x_1 w_1^{[1]} w_1^{[2]} + x_2 w_3^{[1]} w_1^{[2]} + x_1 w_2^{[1]} w_2^{[2]} + x_2 w_4^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times x_2 w_1^{[2]}$$

$$= x_2 y_1^{[1]} w_1^{[2]^2} + x_2 y_2^{[1]} w_1^{[2]} w_2^{[2]} - x_2 \hat{y} w_1^{[2]}$$

$$\therefore \frac{\partial}{\partial w_3^{[1]}} J(y, \hat{y}) = (y - \hat{y}) x_2 w_1^{[2]}$$

④ 오차 역전파(backpropagation)를 통한 가중치 갱신

- 오차 함수를 가중치 $w_3^{[1]}$ 과 $w_4^{[1]}$ 에 대해서 편미분한 도함수를 구하면 다음과 같다.

$$\frac{\partial}{\partial w_4^{[1]}} J(y, \hat{y}) = \frac{\partial}{\partial w_4^{[1]}} \left(\frac{1}{2} (y - \hat{y})^2 \right) = (y - \hat{y}) \times \frac{\partial}{\partial w_4^{[1]}} (y - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_4^{[1]}} (y_1^{[1]} w_1^{[2]} + y_2^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times \frac{\partial}{\partial w_4^{[1]}} (x_1 w_1^{[1]} w_1^{[2]} + x_2 w_3^{[1]} w_1^{[2]} + x_1 w_2^{[1]} w_2^{[2]} + x_2 w_4^{[1]} w_2^{[2]} - \hat{y})$$

$$= (y - \hat{y}) \times x_2 w_2^{[2]}$$

$$= x_2 y_1^{[1]} w_1^{[2]} w_2^{[2]} + x_2 y_2^{[1]} w_2^{[2]^2} - x_2 \hat{y} w_2^{[2]}$$

$$\therefore \frac{\partial}{\partial w_4^{[1]}} J(y, \hat{y}) = (y - \hat{y}) x_2 w_2^{[2]}$$

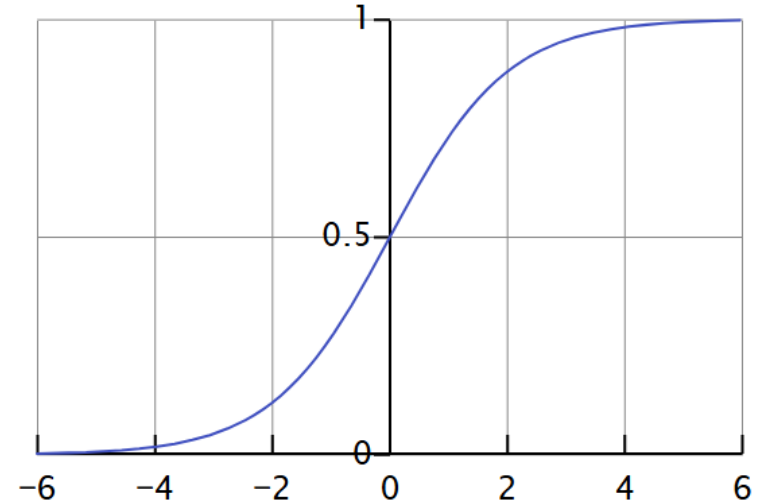
더 생각해볼 문제 (1)

- 실제 딥러닝 모델에서 활성화 함수는 앞서 예제에서 사용한 $f(x) = x$ 를 사용하지 않고, 비선형 함수(nonlinear function)를 활용한다.
- 대표적 비선형 함수로 아래의 Sigmoid function이 있다.

$$f(x) = \frac{1}{1 + \exp(-x)}$$

- 한편, 상기 Sigmoid function의 도함수는 다음과 같다.

$$\frac{d}{dx}f(x) = \frac{d}{dx}\left(\frac{1}{1 + \exp(-x)}\right) = f(x)(1 - f(x))$$



[문제] Sigmoid function $f(x)$ 에서 x 에 대한 미분 결과가 $f(x)(1 - f(x))$ 임을 유도하시오.

더 생각해볼 문제 (2)

- 앞서 설명한 예제는 가중치 갱신 방법을 6개의 가중치 각각에 대한 공식으로 풀어서 설명하였다.
- 그러나 매우 많은 수의 가중치를 갱신할 때, 이러한 가중치 갱신 방법을 for 반복문을 이용해서 구현한다면 수행 시간이 급격하게 증가하는 문제점이 발생한다.
- 따라서, 앞서 설명한 가중치 갱신 방법은 벡터와 행렬을 이용하여, 벡터와 행렬의 곱셈 연산으로 표현할 수 있는데, 이를 **vectorization**이라 한다.
 - Vectorization에 대해서는 다음 수업 시간에 설명 예정

Q & A